



C++ - Módulo 01

Asignación de memoria, punteros a miembros,
referencias y declaraciones switch

Resumen:

Este documento contiene los ejercicios del Módulo 01 de los módulos de C++.

Versión: 11.1

Contenido

I	Introducción	2
II	Reglas generales	3
III	Instrucciones de IA	6
IV	Ejercicio 00: Cerebrooooooooooooo	8
V	Ejercicio 01: ¡Más cerebros!	9
VI	Ejercicio 02: HOLA ESTO ES CEREBRO	10
VII	Ejercicio 03: Violencia innecesaria	11
VIII	Ejercicio 04: Sed es para perdedores	13
IX	Ejercicio 05: Harl 2.0	14
X	Ejercicio 06: Filtro Harl	16
XI	Presentación y evaluación por pares	17

Capítulo I

Introducción

C++ es un lenguaje de programación de propósito general creado por Bjarne Stroustrup como una extensión del lenguaje de programación C, o "C con clases" (fuente: [Wikipedia](#)).

El objetivo de estos módulos es introducirte en la Programación Orientada a Objetos. Este será el punto de partida de tu experiencia con C++. Se recomiendan muchos lenguajes para aprender programación orientada a objetos. Hemos elegido C++ porque deriva de tu viejo amigo, C. Debido a que se trata de un lenguaje complejo y para mantener las cosas simples, su código cumplirá con el estándar C++98.

Sabemos que el C++ moderno es muy diferente en muchos aspectos. Así que, si quieres convertirte en un desarrollador competente de C++, ¡de ti depende ir más allá de los 42 estándares Common Core!

Capítulo II

Reglas generales

Compilando

- Compila tu código con `c++` y los indicadores `-Wall -Wextra -Werror`
- Su código aún debería compilarse si agrega el indicador `-std=c++98`

Convenciones de formato y nomenclatura

- Los directorios de ejercicios se nombrarán de esta manera: `ex00`, `ex01`, ... , `exn`
- Nombra tus archivos, clases, funciones, funciones miembro y atributos según sea necesario en las directrices.
- Escriba los nombres de clase en formato Mayúsculas y minúsculas . Los archivos que contienen código de clase siempre se nombrarán según el nombre de la clase.
Por ejemplo: `ClassName.hpp/ClassName.h`, `ClassName.cpp` o `ClassName.hpp`. Si tiene un archivo de encabezado que contiene la definición de la clase "BrickWall", que representa un muro de ladrillos, su nombre será `BrickWall.hpp`.
- A menos que se especifique lo contrario, cada mensaje de salida debe terminar con un carácter de nueva línea y mostrarse en la salida estándar.

¡ Adiós, Norminette! No se impone ningún estilo de programación en los módulos de C++. Puedes seguir tu favorito. Pero recuerda que el código que tus pares evaluadores no pueden entender es código que no pueden calificar. Haz todo lo posible por escribir código limpio y legible.

Permitido/Prohibido

Ya no estás programando en C. ¡Hora de usar C++! Por lo tanto:

- Puedes usar casi todo lo de la biblioteca estándar. Por lo tanto, en lugar de limitarte a lo que ya sabes, sería inteligente usar las versiones C++ de las funciones de C que conoces siempre que sea posible.

Sin embargo, no puedes usar ninguna otra biblioteca externa. Esto significa que C++11 (y sus derivados) y las bibliotecas de Boost están prohibidas. Las siguientes funciones también están prohibidas: `*printf()`, `*alloc()` y `free()`. Si las usas, tu calificación será 0 y punto.

- Tenga en cuenta que, a menos que se indique explícitamente lo contrario, el uso del espacio de nombres `<ns_name>` y Las palabras clave de amistad están prohibidas. De lo contrario, tu calificación será -42.
- Solo se permite usar el STL en los módulos 08 y 09. Esto significa: no se permiten contenedores (vectoriales, de lista, de mapa, etc.) ni algoritmos (cualquier cosa que requiera incluir el encabezado `<algorithm>`) hasta entonces. De lo contrario, su calificación será de -42.

Algunos requisitos de diseño

- La pérdida de memoria también ocurre en C++. Cuando se asigna memoria (mediante el nuevo palabra clave), debe evitar fugas de memoria.
- Desde el Módulo 02 hasta el Módulo 09, sus clases deben estar diseñadas en el modelo Ortodoxo. Forma canónica, salvo que se indique explícitamente lo contrario.
- Cualquier implementación de función colocada en un archivo de encabezado (excepto las plantillas de función) significa 0 para el ejercicio.

Debes poder usar cada encabezado independientemente. Por lo tanto, deben incluir todas las dependencias necesarias. Sin embargo, debes evitar el problema de la doble inclusión añadiendo protecciones de inclusión. De lo contrario, tu calificación será 0.

Léame

- Puedes agregar archivos adicionales si lo necesitas (por ejemplo, para dividir tu código). Dado que estas tareas no están verificadas por un programa, puedes hacerlo siempre y cuando entregues los archivos obligatorios.
- A veces, las pautas de un ejercicio parecen cortas pero los ejemplos pueden mostrar requisitos que no están escritos explícitamente en las instrucciones.
- ¡Lee cada módulo completo antes de empezar! ¡De verdad, hazlo!

¡Por Odín, por Thor! ¡Usa tu cerebro!



Respecto al Makefile para proyectos C++, se aplican las mismas reglas que en C (ver el capítulo de Normas sobre el Makefile).



Tendrás que implementar muchas clases. Esto puede parecer tedioso, a menos que puedas programar tu editor de texto favorito.



Se le da una cierta cantidad de libertad para completar los ejercicios.
Sin embargo, sigue las reglas obligatorias y no seas perezoso.
¡Te pierdes mucha información útil! No dudes en leer sobre...
conceptos teóricos.

Capítulo III

Instrucciones de IA

- Context

Este proyecto está diseñado para ayudarle a descubrir los componentes fundamentales de su entrenamiento 42.

Para anclar adecuadamente los conocimientos y habilidades clave, es esencial adoptar un enfoque reflexivo al utilizar las herramientas y el soporte de IA.

El verdadero aprendizaje fundamental requiere un esfuerzo intelectual genuino, a través del desafío, la repetición y los intercambios de aprendizaje entre pares.

Para obtener una descripción más completa de nuestra postura sobre la IA (como herramienta de aprendizaje, como parte de la capacitación 42 y como una expectativa en el mercado laboral), consulte las preguntas frecuentes específicas en la intranet.

- Mensaje principal

Construye bases sólidas sin atajos.

Desarrollar realmente habilidades técnicas y de potencia.

Experimente el verdadero aprendizaje entre pares, comience a aprender a aprender y a resolver nuevos problemas.

El recorrido de aprendizaje es más importante que el resultado.

Conozca los riesgos asociados con la IA y desarrolle prácticas de control efectivas y contramedidas para evitar errores comunes.

- Reglas del alumno:

- Debes aplicar el razonamiento a las tareas asignadas, especialmente antes de recurrir a la IA.

- No debes pedir respuestas directas a la IA.
- Debes conocer los 42 enfoques globales sobre IA.

• Resultados de la fase:

Dentro de esta fase fundamental, obtendrás los siguientes resultados:

- Obtenga bases técnicas y de codificación adecuadas.
- Conozca por qué y cómo la IA puede ser peligrosa durante esta fase.

• Comentarios y ejemplo:

Sí, sabemos que la IA existe, y sí, puede resolver tus proyectos. Pero estás aquí para aprender, no para demostrar que la IA ha aprendido. No pierdas tu tiempo (ni el nuestro) solo para demostrar que la IA puede resolver el problema.

Aprender a los 42 años no se trata de saber la respuesta, sino de desarrollar la capacidad de encontrarla. La IA te da la respuesta directamente, pero eso te impide desarrollar tu propio razonamiento. Y razonar requiere tiempo, esfuerzo e implica fracaso. El camino al éxito no se supone que sea fácil.

- Tenga en cuenta que durante los exámenes, la IA no está disponible (no hay Internet, ni teléfonos inteligentes, etc.). Rápidamente se dará cuenta si ha confiado demasiado en la IA en su proceso de aprendizaje.

El aprendizaje entre pares te expone a diferentes ideas y enfoques, mejorando tus habilidades interpersonales y tu capacidad de pensar de forma divergente. Esto es mucho más valioso que simplemente chatear con un bot. Así que no seas tímido: ¡habla, pregunta y aprende con otros!

Sí, la IA formará parte del plan de estudios, tanto como herramienta de aprendizaje como tema en sí mismo. Incluso tendrás la oportunidad de crear tu propio software de IA. Para obtener más información sobre nuestro enfoque progresivo, consulta la documentación disponible en la intranet.

Buenas prácticas:

Estoy atascado con un nuevo concepto. Le pregunto a alguien cercano cómo lo abordó. Hablamos durante 10 minutos y, de repente, todo encaja. Lo entiendo.

Mala práctica:

Uso IA en secreto y copio código que parece correcto. Durante la evaluación por pares, no puedo explicar nada. Suspenso. Durante el examen, sin IA, me quedo atascado de nuevo. Suspenso.

Capítulo IV

Ejercicio 00: Cerebrooooooooooooo

	Ejercicio: 00
Cerebrooooooooooooo	
Directorio: ex00/	
Archivos a enviar: Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, newZombie.cpp, randomChump.cpp Prohibido: Ninguno	

Primero, implementa una clase `Zombie`. Su atributo de cadena privado es "name".

Agregue una función miembro `void Announcement( void );` a la clase `Zombie`. Zombies

Se anuncian de la siguiente manera:

<nombre>: Cerebrooooooooooooo...

No imprima los corchetes angulares (< y >). Para un zombi llamado Foo, el mensaje sería:

Foo: Cerebrooooooooooooo...

Luego, implemente las siguientes dos funciones:

- `Zombie* newZombie( std::string name );` Esta función crea un zombie, lo nombra y lo devuelve para que puedas usarlo fuera del alcance de la función.
- `void randomChump( std::string nombre );`
Esta función crea un zombie, le pone un nombre y hace que se anuncie a sí mismo.

Ahora bien, ¿cuál es el verdadero objetivo del ejercicio? Hay que determinar en qué caso Es mejor asignar zombies en la pila o el montón.

Los zombis deben destruirse cuando ya no los necesites. El destructor debe imprimir un mensaje con el nombre del zombie para fines de depuración.

Capítulo V

Ejercicio 01: ¡Buenos días cerebros!

	Ejercicio: 01
¡Más cerebros!	
Directorio: ex01/	
Archivos a enviar: Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, zombieHorde.cpp Prohibido:	
Ninguno	

¡Es hora de crear una horda de zombis!

Implemente la siguiente función en el archivo apropiado:

```
Zombie* zombieHorde( int N, std::string nombre );
```

Debe asignar N objetos Zombie en una sola asignación. Luego, debe inicializar los zombies, asignando a cada uno el nombre pasado como parámetro. La función devuelve un puntero al primer zombie.

Implementa tus propias pruebas para asegurar que tu función zombieHorde() funcione como se indica. esperado. Intenta llamar a Announce() para cada uno de los zombies.

No olvides usar la tecla eliminar para desasignar todos los zombies y verificar si hay fugas de memoria.

Capítulo VI

Ejercicio 02: HOLA ESTO ES CEREBRO

	Ejercicio: 02
HOLA ESTE ES CEREBRO	
Directorio: ex02/	
Archivos a enviar: Makefile, main.cpp Prohibido:	
Ninguno	

Escriba un programa que contenga:

- Una variable de cadena inicializada como "HOLA ESTO ES CEREBRO".
- stringPTR: un puntero a la cadena.
- stringREF: una referencia a la cadena.

Su programa debe imprimir:

- La dirección de memoria de la variable de cadena.
- La dirección de memoria contenida en stringPTR.
- La dirección de memoria contenida en stringREF.

Y luego:

- El valor de la variable de cadena.
- El valor señalado por stringPTR.
- El valor al que apunta stringREF.

Eso es todo, sin trucos. El objetivo de este ejercicio es desmitificar las referencias, que pueden parecer completamente nuevas. Aunque hay algunas pequeñas diferencias, se trata simplemente de otra sintaxis para algo que ya se hace: la manipulación de direcciones.

Capítulo VII

Ejercicio 03: Violencia innecesaria

	Ejercicio: 03
Violencia innecesaria	
Directorio: ex03/	
Archivos a enviar: Makefile, main.cpp, Weapon.{h, hpp}, Weapon.cpp, HumanA.{h, hpp}, HumanA.cpp, HumanB.{h, hpp}, HumanB.cpp Prohibido: Ninguno	

Implementar una clase de arma que tenga:

- Un tipo de atributo privado, que es una cadena.
- Una función miembro getType() que devuelve una referencia constante al tipo.
- Una función miembro setType() que establece el tipo utilizando el nuevo valor pasado como parámetro.

Ahora, crea dos clases: HumanA y HumanB. Ambas tienen un arma y un nombre. También tienen una función miembro `attack()` que muestra (sin los corchetes angulares):

<nombre> ataca con su <tipo de arma>

HumanA y HumanB son casi idénticos excepto por estos dos pequeños detalles:

- Mientras que HumanA toma el Arma en su constructor, HumanB no lo hace.
- HumanB puede no tener siempre un arma, mientras que HumanA siempre la tendrá .
armado.

Si su implementación es correcta, al ejecutar el siguiente código se imprimirá un ataque con "garrote con púas tosco" seguido de un segundo ataque con "algún otro tipo de garrote" para ambos casos de prueba:

```
int principal()
{
    {
        Club de armas = Arma(" club de púas tosco");

        HumanA bob("Bob", club);
        bob.attack();
        club.setType("algún otro tipo de club"); bob.attack(); } {

        Club de armas = Arma(" club de púas tosco");

        HumanB jim("Jim");
        jim.setWeapon(club);
        jim.attack();
        club.setType("algún otro tipo de club"); jim.attack();

    }

    devuelve 0;
}
```

No olvides comprobar si hay fugas de memoria.



¿En qué caso crees que sería mejor usar un puntero a Arma? ¿Y una referencia a Arma? ¿Por qué? Piénsalo antes de empezar este ejercicio.

Capítulo VIII

Ejercicio 04: Sed es para perdedores

	Ejercicio: 04
Sed es para perdedores	
Directorio: ex04/	
Archivos a enviar: Makefile, main.cpp, *.cpp, *.h, hpp}	
Prohibido: std::string::replace	

Cree un programa que tome tres parámetros en el siguiente orden: un nombre de archivo y dos cuerdas, s1 y s2.

Debe abrir el archivo <filename> y copiar su contenido en un nuevo archivo <filename>.replace, reemplaza cada ocurrencia de s1 con s2.

El uso de funciones de manipulación de archivos de C está prohibido y se considerará trampa. Se permiten todas las funciones miembro de la clase std::string, excepto replace. ¡Úsalas con cuidado!

Por supuesto, gestiona las entradas y los errores inesperados. Debes crear y entregar tu Realice sus propias pruebas para garantizar que su programa funcione como se espera.

Capítulo IX

Ejercicio 05: Harl 2.0

	Ejercicio: 05
	Harl 2.0
	Directorio: ex05/
	Archivos a enviar: Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp Prohibido: Ninguno

¿Conoces a Harl? Todos lo conocemos, ¿verdad? Si no, a continuación encontrarás los tipos de comentarios que hace Harl. Están clasificados por niveles:

- Nivel "DEBUG" : Los mensajes de depuración contienen información contextual. Se utilizan principalmente para el diagnóstico de problemas.
Ejemplo: "Me encanta tener tocino extra para mi hamburguesa 7XL con doble queso, triple pepinillo y ketchup especial. ¡De verdad!"
- Nivel "INFO" : Estos mensajes contienen información extensa. Son útiles para Seguimiento de la ejecución del programa en un entorno de producción.
Ejemplo: "No puedo creer que añadir tocino extra cueste más dinero. ¡No le pusiste suficiente tocino a mi hamburguesa! Si lo hubieras hecho, ¡no pediría más!"
- Nivel "ADVERTENCIA" : Los mensajes de advertencia indican un problema potencial en el sistema.
Sin embargo, se puede manejar o ignorar.
Ejemplo: "Creo que merezco un poco de tocino extra gratis. Llevo viniendo años, mientras que tú empezaste a trabajar aquí el mes pasado".
- Nivel "ERROR" : Estos mensajes indican que se ha producido un error irreparable.
Generalmente se trata de un problema crítico que requiere intervención manual.
Ejemplo: "¡Esto es inaceptable! Quiero hablar con el gerente ahora mismo".

Vas a automatizar a Harl. No será difícil, ya que siempre dice lo mismo.
Cosas. Debes crear una clase Harl con las siguientes funciones miembro privadas:

- void debug(vacío);
- información vacía (vacío);
- advertencia de vacío (void);
- error nulo(void);

Harl también tiene una función miembro pública que llama a las cuatro funciones miembro anteriores dependiendo del nivel pasado como parámetro:

```
vacío    quejarse( std::string nivel );
```

El objetivo de este ejercicio es usar punteros a funciones miembro. Esto no es una sugerencia. Harl tiene que quejarse sin usar un bosque de if/else. ¡No lo piensa dos veces!

Crea y entrega pruebas para demostrar que Harl se queja mucho. Puedes usar los ejemplos.
de los comentarios enumerados anteriormente en el asunto o elija utilizar comentarios propios.

Capítulo X

Ejercicio 06: Filtro Harl

	Ejercicio: 06
	Filtro Harl
	Directorio: ex06/
	Archivos a enviar: Makefile, main.cpp, Harl.{h, cpp}, Harl.cpp Prohibido: Ninguno

A veces no quieres prestar atención a todo lo que dice Harl. Implementa un sistema para filtrar lo que dice Harl dependiendo de los niveles de registro que quieras escuchar.

Cree un programa que tome como parámetro uno de los cuatro niveles. Mostrará todos los mensajes de este nivel y superiores. Por ejemplo:

```
$> ./harlFilter "ADVERTENCIA"
[ ADVERTENCIA ]
Creo que merezco tener un poco de tocino extra gratis.
Llevo viniendo años, mientras que tú empezaste a trabajar aquí el mes pasado.

[ ERROR ]
¡Esto es inaceptable! Quiero hablar con el gerente ahora.

$> ./harlFilter "No estoy seguro de lo cansado que estoy hoy..."
[Probablemente quejándose de problemas insignificantes]
```

Aunque hay varias formas de lidiar con Harl, una de las más efectivas es APAGARLO.

Dale el nombre harlFilter a tu ejecutable.

Debes utilizar, y tal vez descubrir, la declaración switch en este ejercicio.



Puedes aprobar este módulo sin realizar el ejercicio 06.

Capítulo XI

Presentación y evaluación por pares

Entregue su tarea en su repositorio de Git como de costumbre. Solo se evaluará el trabajo dentro de su repositorio durante la defensa. No dude en verificar los nombres de sus carpetas y archivos para asegurarse de que sean correctos.

Durante la evaluación, ocasionalmente se podría solicitar una breve modificación del proyecto . Esto podría implicar un cambio menor de comportamiento, la escritura o reescritura de algunas líneas de código, o una función fácil de añadir.

Si bien este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar su comprensión real de una parte específica del proyecto.

La modificación se puede realizar en cualquier entorno de desarrollo que elija (por ejemplo, su configuración habitual) y debería ser factible en unos pocos minutos, a menos que se defina un período de tiempo específico como parte de la evaluación.

Por ejemplo, se le puede pedir que realice una pequeña actualización a una función o script, modifique una pantalla o ajuste una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.